ZENITH STUDIO LLC

SMART CONTRACT SECURITY ASSESSMENT

NEWERA-AI (NEA)

$NEA TOKEN

SMART CONTRACT SECURITY AUDIT REPORT

GENERATE, SELL IMAGES OR PROMPTS. EVERY ACTION IS ON-CHAIN.

SECURITY ASSESSMENT

Source-Code Review

BNB Smart Chain

ASSESSMENT PROVIDER

ZENITH STUDIO LLC

License No. 17003626

ADShoshada Road, Ras Al Khaimah,

United Arab Emirates

Report Version: 1.0 *Assessment Date: 15.10.2026* *Status: Final*

Assessment status

PASS - No Critical, High, Medium or Low severity code findings identified from the reviewed custom contract. Two informational observations are documented.

Field	Value
Project	NewEra
Token / Symbol	$NEA
Network	BNB Smart Chain (BSC)
Contract address	0x562BFE0Eb9180928665cfe67c97Bc410964dd014
Contract name	NewEra NEA
Decimals	18
Initial supply	30B
Source pragma	v0.8.24+commit.e11b9ed9
Reported deployment compiler	v0.8.24+commit.e11b9ed9
Framework	OpenZeppelin Contracts (ERC20, Ownable)
License	MIT
Report version	1
Report date	Oct 2026
Assessment type	Manual source-code review (point-in-time)
Prepared for	NewEra



1. Disclaimer and Legal Notice

This report presents a point-in-time, source-code-focused security assessment of the NewEra (NEA) token contract, based solely on the materials supplied by the project team and enumerated in the scope section. It is a technical review and does not constitute financial, investment, legal, or tax advice, nor a recommendation to buy, hold, or sell any asset.

2. Executive Summary

The NewEra contract is a minimal ERC-20 token that inherits its core behaviour from the OpenZeppelin ERC20 and Ownable libraries and adds a very small amount of custom logic: a public supply constant, a constructor that mints a fixed supply, and a holder-only burn function. The custom attack surface is therefore small.

Within the defined scope, no Critical, High, or Medium severity issues were identified in the custom contract code. The contract exposes no post-deployment minting path, no transfer fee or tax, no blacklist or whitelist, no pausing or freezing mechanism, and no upgradeability. The owner role, as implemented, carries no token-level authority beyond standard ownership transfer and renouncement.

Subject to those caveats, the custom code presents a LOW CUSTOM-CODE RISK PROFILE.

3. Audit Scope

3.1 In scope

The single Solidity source file for the NewEra contract, as supplied by the project team (reproduced in Appendix A). The custom logic declared in that contract: the INITIAL_SUPPLY constant, the constructor, and the burn function. The declared inheritance from OpenZeppelin ERC20 and Ownable, at the interface/behaviour level.

3.2 Out of scope

The internal implementation of the OpenZeppelin libraries, which are widely used and independently published, and were not re-audited here. Independent verification that the deployed on-chain bytecode matches the supplied source (see Section 15). Tokenomics, emissions, reward mechanics, liquidity, and any economic or game-design behaviour. Off-chain systems: the game client and servers, reward backends, APIs, key management, and deployment operations. Front-end, website, and any contracts other than the one at the address above.

3.3 Materials provided

Project and deployment information (network, address, name, symbol, supply, decimals, compiler details).The full Solidity source of the NewEra contract. A team statement that the contract has been verified on BscScan (not independently confirmed here).



4. Methodology

The assessment was performed by manual review of the supplied Solidity source. Each function and state element was examined against common smart-contract weakness classes and ERC-20 best practices, including access control, minting authority, arithmetic safety, reentrancy, external-call safety, and the presence of privileged or hidden administrative controls.

Severity	Definition (as used in this report)
Critical	Directly leads to loss of funds, unauthorized minting, or contract takeover.
High	Serious impact under realistic conditions; may require specific preconditions.
Medium	Meaningful risk or deviation from best practice with limited direct impact.
Low	Minor issue, hardening opportunity, or centralization/distribution consideration.
Informational	Observation, clarity, or documentation note with no direct security impact.

5. Contract Architecture

The contract is declared as: contract NewEra is ERC20, Ownable. It combines standard fungible-token behaviour (ERC20) with a single-owner access-control primitive (Ownable). The custom code footprint is limited to three elements, with all remaining behaviour inherited.

5.1 Inherited vs. custom functionality

Component	Origin	Notes
transfer, transferFrom, approve, allowance, balanceOf, totalSupply, name, symbol, decimals	Inherited (OpenZeppelin ERC20)	Standard ERC-20 semantics; not overridden.
_mint, _burn, _update (internal)	Inherited (OpenZeppelin ERC20)	Internal accounting primitives.
owner, transferOwnership, renounceOwnership, onlyOwner	Inherited (OpenZeppelin Ownable)	Ownership management only.
INITIAL_SUPPLY (constant)	Custom	Public constant = 10,000,000,000 (unscaled).
constructor(address initialHolder)	Custom	Sets owner and mints the fixed supply once.
burn(uint256 amount)	Custom	Holder-only burn of caller balance.



6. Detailed Analysis

6.1 Token supply model

Fixed. The constructor calls _mint(initialHolder, INITIAL_SUPPLY * 10 ** decimals()) exactly once. No other call site invokes _mint, and the contract exposes no external or owner-callable minting function. Total supply is therefore fixed at deployment and can only decrease through burning. This removes inflation and unauthorized-mint risk from the custom code.

6.2 ERC-20 implementation

Standard and unmodified. The contract does not override transfer logic (no override of _update, _transfer, or the public transfer functions). ERC-20 behaviour is inherited verbatim from OpenZeppelin, so transfers, approvals, and balances follow the reference implementation.

6.3 Constructor

Runs once at deployment. It requires initialHolder != address(0), passes initialHolder to Ownable (setting the owner), and mints the full supply to initialHolder. The explicit zero-address check is defensive and partially redundant, as OpenZeppelin Ownable and _mint also reject the zero address.

6.4 Ownership model

OpenZeppelin Ownable with a constructor-supplied initial owner. Critically, no function in the custom code is guarded by onlyOwner, and no inherited token function is owner-restricted. The owner can therefore only call transferOwnership and renounceOwnership; it has no ability to mint, pause, freeze, tax, blacklist, or move other users' tokens.

6.5 Burn mechanism

burn(uint256 amount) calls _burn(msg.sender, amount), which burns only the caller's own balance and reverts on insufficient balance. There is no burnFrom and no ability to burn third-party tokens. The function reduces total supply and is available to any holder.

6.6 Transfer functionality

No custom transfer restrictions exist. Because transfer logic is not overridden, there is no fee, no maximum-transaction or wallet cap, no cooldown, no blacklist, and no pause affecting transfers. Transfers behave as standard ERC-20 transfers.

6.7 Approval / allowance

Standard OpenZeppelin approve / allowance / transferFrom. Note that OpenZeppelin v5 (indicated by the Ownable(initialOwner) constructor signature) does not include increaseAllowance / decreaseAllowance helpers; the well-known ERC-20 approve front-running characteristic is inherent to the standard and is noted as informational (I-04).

6.8 Access control

Limited to Ownable. No role-based access control (AccessControl) and no custom modifiers are present. There are no privileged token operations to gate.

6.9 Arithmetic safety

The pragma is ^0.8.24 (reported deployment 0.8.28), so arithmetic is checked for overflow/underflow by the compiler. The custom code performs only one multiplication (INITIAL_SUPPLY * 10 ** decimals()), evaluated once at deployment with fixed constants. No custom unchecked blocks are present.



6.10 Reentrancy

No reentrancy vector was identified in the custom code. There are no external calls, and OpenZeppelin v5 ERC20 does not invoke transfer hooks or token-receiver callbacks during transfers or burns. The burn function performs a single internal state update.

6.11 External calls

None in the custom code. There are no low-level call, delegatecall, static call, send, or transfer-of-ETH operations, and no interaction with external contracts.

6.12 Minting authority

No post-deployment minting authority exists. _mint is reachable only from the constructor; there is no mint, ownerMint, or equivalent function.

6.13 Balance manipulation

No function permits arbitrary setting of balances. Balances change only through inherited transfer/transferFrom and the holder-only burn.

6.14 Blacklist / whitelist

Not present. No address is granted or denied transfer rights, and no allow/deny list logic exists in the source.

6.15 Transfer tax / fee

Not present. No fee, tax, or redistribution logic exists; transfers move the full amount.

6.16 Pausing / freezing

Not present. The contract does not inherit Pausable and defines no pause, freeze, or trading-enable switch.

6.17 Upgradeability

Not upgradeable. The contract is a standard (non-proxy) deployment with no delegatecall, no upgrade function, and no storage-proxy pattern. The deployed logic is immutable.

6.18 Hidden administrative controls

None were identified in the supplied source. The owner holds no token-level powers. This conclusion is limited to the reviewed source; equivalence with the deployed bytecode is addressed in Section 15 and is not independently verified here.

6.19 Denial-of-service

No DoS vector was identified in the custom code. There are no unbounded loops, no iteration over user-controlled arrays, and no dependence on external calls that could be made to fail.



7. Findings

The following findings summarize the assessment. No Critical, High, or Medium severity findings were identified in the custom contract code. One Low and five Informational items are recorded.

ID	Title	Severity	Status
L-01	Initial supply concentration in a single holder	**Low**	Open / By design
I-01	OpenZeppelin dependency version not confirmed or pinned	**Informational**	Open
I-02	Unscaled INITIAL_SUPPLY constant may cause confusion	**Informational**	Open
I-03	Redundant zero-address validation in constructor	**Informational**	Open
I-04	ERC-20 approve race condition (standard characteristic)	**Informational**	Acknowledged
I-05	Ownership retained without owner-gated functionality	**Informational**	Open

7.1 Detailed findings

L-01 — Initial supply concentration in a single holder

Severity	Low	Status	Open / By design

Description. The entire initial supply (30,000,000,000 NEA) is minted to a single address (initialHolder) at deployment, which is also set as the contract owner. Concentration of the full supply in one address is a distribution and market consideration rather than a code-execution vulnerability.

Basis in code. The constructor executes _mint(initialHolder, INITIAL_SUPPLY * 10 ** decimals()) and Ownable(initialHolder). Subsequent distribution (for example to liquidity, treasury, or vesting) occurs off-chain and cannot be verified from the source.

Recommendation. Distribute the supply according to a published allocation, and where appropriate use time-locked or vesting arrangements for team and treasury holdings. Publish the holder address so the community can independently track distribution. The distribution itself is Not independently verified from the provided materials.

I-01 — OpenZeppelin dependency version not confirmed or pinned

Severity	Informational	Status	Open

Description. The exact OpenZeppelin Contracts version is not stated in the supplied materials. The Ownable(initialOwner) constructor signature indicates OpenZeppelin v5.x, but the precise release is not confirmed.



Basis in code. The imports reference @openzeppelin/contracts without a version, and no lockfile or build manifest was provided.

Recommendation. Pin a specific, released OpenZeppelin version in the build configuration and record it in project documentation so third parties can reproduce the build. The exact version is Not independently verified from the provided materials.

I-02 — Unscaled INITIAL_SUPPLY constant may cause confusion

Severity	Informational	Status	Open

Description. The public constant INITIAL_SUPPLY returns 10,000,000,000 (without decimals), whereas the actual minted and reported total supply is that value multiplied by 10^18. Readers querying the constant directly may misinterpret the figure.

Basis in code. INITIAL_SUPPLY is declared as 30_000_000_000 and scaled only at mint time via * 10 ** decimals().

Recommendation. Document clearly that the on-chain total supply is 10,000,000,000 × 10^18 base units, or expose a pre-scaled value, to avoid confusion for integrators and explorers.

I-03 — Redundant zero-address validation in constructor

Severity	Informational	Status	Open

Description. The constructor includes require(initialHolder != address(0), "Invalid holder"). OpenZeppelin Ownable and _mint already revert on the zero address, making this check partially redundant.

Basis in code. The explicit require duplicates protections already present in the inherited constructor and mint path.

Recommendation. No action is required; the check is harmless and defensive. It may be retained for explicitness.

I-04 — ERC-20 approve race condition (standard characteristic)

Severity	Informational	Status	Acknowledged

Description. The standard ERC-20 approved pattern carries a well-known allowance front-running characteristic. This is inherent to the ERC-20 standard and not specific to NewEra. OpenZeppelin v5 does not provide increaseAllowance / decreaseAllowance helpers.

Basis in code. Approval logic is inherited unchanged from OpenZeppelin ERC20.

Recommendation. Integrators should follow standard practice (set allowance to zero before setting a new non-zero value where relevant). No contract change is necessary.



I-05 — Ownership retained without owner-gated functionality

Severity	Informational	Status	Open

Description. The contract retains an owner, but no function grants the owner any token-level control. The residual owner surface is limited to transferOwnership and renounceOwnership.

Basis in code. No onlyOwner-guarded function exists in the custom or inherited token operations used by this contract.

Recommendation. If no future owner-gated functionality is intended, consider renouncing ownership after deployment and initial setup to remove the residual owner surface. Whether ownership has been renounced on-chain is Not independently verified from the provided materials.

8. Centralization and Privilege Analysis

The table below enumerates privileged capabilities present in the reviewed source.

Role	Capabilities in this contract	Token-level control?
owner()	transferOwnership, renounceOwnership	None
initialHolder	Holds 100% of supply at deployment (as a token holder, not via privilege)	Standard holder rights only
Any holder	transfer, approve, transferFrom (own/allowed), burn (own balance)	Standard ERC-20 only

The owner cannot mint, pause, freeze, tax, blacklist, or seize tokens, because no such function exists in the source. The principal centralization consideration is the initial concentration of supply and the off-chain distribution that follows, not any on-chain administrative power.

9. Deployment and Configuration Observations

Compiler - The source pragma is ^0.8.24, which permits compilation with a range of 0.8.x compilers; the reported deployment compiler is 0.8.28. Both include built-in overflow checks. Pinning an exact pragma is recommended for reproducibility.

Constructor parameter - The security of the initial distribution depends on the initialHolder address supplied at deployment. The specific address used is Not independently verified from the provided materials.



10. Dependency Risk

The contract relies on the OpenZeppelin ERC20 and Ownable libraries. These are widely used, publicly available implementations and were not re-audited as part of this assessment. The exact version was not provided; the Ownable(initialOwner) signature indicates OpenZeppelin v5.x. Pinning and recording the exact released version is recommended (see I-01).

11. Final Conclusion

Based on the source code and deployment information reviewed within the defined scope, no Critical, High, or Medium severity vulnerabilities were identified in the custom NewEra contract code. This conclusion follows from the specific properties of the code: the supply is fixed and minted once, with no post-deployment minting path; transfers, approvals, and balances are inherited unchanged from OpenZeppelin with no fee, tax, blacklist, pause, or cap; the owner role confers no token-level authority; there are no external calls, no delegate calls, and no upgradeability; and arithmetic is compiler-checked.

12. Audit Summary

Severity	Findings
Critical	0
High	0
Medium	0
Low	1
Informational	5

Field	Value
Overall assessment	Low custom-code risk profile (subject to stated caveats)
Contract	0x562BFE0Eb9180928665cfe67c97Bc410964dd014
Network	BNB Smart Chain
Token	NewEra (NEA)



13. Categorized Results

13.1 Source-code findings

1. L-01 Initial supply concentration in a single holder (Low).
2. I-02 Unscaled INITIAL_SUPPLY constant (Informational).
3. I-03 Redundant zero-address validation (Informational).
4. I-04 Standard ERC-20 approve race condition (Informational).
5. I-05 Ownership retained without owner-gated functionality (Informational).

13.4 Items outside the audit scope

Internal implementation of the OpenZeppelin libraries. Tokenomics, rewards, emissions, liquidity, and economic design. Off-chain systems: game client/servers, reward backends, APIs, key management, and operations. Independent on-chain verification and dynamic testing (fuzzing, formal verification, penetration testing).

Positive Security Properties

- No externally callable mint function.
- No hidden tax or fee mechanism in the reviewed source.
- No blacklist or whitelist mechanism.
- No pause / freeze mechanism.
- No arbitrary balance modification function.
- Burn restricted to caller's own balance.
- No external calls in project-specific functions.
- No loops in project-specific functions.
- No proxy or upgrade mechanism.
- Standard OpenZeppelin ERC20 implementation.
- Standard OpenZeppelin Ownable implementation.

Appendix A – Reference & Basis

That reference report uses contract details, function details, security checks, owner privileges, severity-based findings and a conclusion. Its own disclaimer limits conclusions to the audited contract and does not assess business-model attractiveness or other contracts.

OpenZeppelin documentation describes ERC20 as the standard fungible-token implementation and documents standard functions including totalSupply, balanceOf, transfer, allowance, approve and transferFrom. OpenZeppelin also documents Ownable as a basic single-owner access-control mechanism.

Appendix B – Source Review Snapshot

Parameter	Reviewed value
Compiler pragma	^0.8.24
Project compiler used	0.8.28
Token name	NewEra
Token symbol	NEA
Initial supply	30,000,000,000
Decimals	18
Constructor parameter	address initialHolder
Initial owner	initialHolder
Initial token recipient	initialHolder
Mint authority after deployment	None exposed
Burn authority	Token holder / msg.sender only

Appendix C – Disclaimer

This document is a security assessment prepared from source code and deployment information made available for review. It is not a certification, guarantee, warranty, legal opinion, financial opinion or substitute for an independent professional audit. Any modification to source, compiler settings, dependencies, deployment address or connected contracts may invalidate portions of this assessment.

